📖 KITAB SAKTI UKK RPL: BEDAH TOTAL KODE TO-DO LIST BERBOBOT LARAVEL

> **Ditulis Khusus Untuk Siswa SMK Rekayasa Perangkat Lunak (RPL)**
> *Panduan Dari Nol: Mengubah Sintaks Rumit Menjadi Cerita Sehari-Hari yang Gampang Dihafal dan Dipahami di Hadapan Penguji.*

DAFTAR ISI

1. BAB 1: GAMBARAN BESAR (MENTAL MODEL ORANG AWAM)
2. BAB 2: PROSES PEMBUATAN DARI HULU KE HILIR
3. BAB 3: BEDAH FILE DAN KODE SECARA MENDETAIL
    - A. File Migrasi Database
    - B. File Seeder (Data Awal)
    - C. File Model & Relasi Eloquent
    - D. File Routing & Satpam Akses (Middleware)
    - E. File Controller (Otak Aplikasi)
    - F. File Tampilan (Blade Views)
4. BAB 4: ALUR PERJALANAN DATA (STUDI KASUS NYATA)
5. BAB 5: KAMUS ISTILAH ASING KE BAHASA MANUSIA (GLOSARIUM)

BAB 1: GAMBARAN BESAR (MENTAL MODEL ORANG AWAM)

Sebelum kita melihat baris-baris kode yang bikin pusing, mari kita samakan frekuensi pikiran terlebih dahulu. Komputer itu sebenarnya makhluk yang sangat polos: dia hanya melakukan apa yang disuruh secara berurutan.

---

1. Apa itu Framework Laravel?

Bayangkan kamu ingin membangun sebuah rumah:

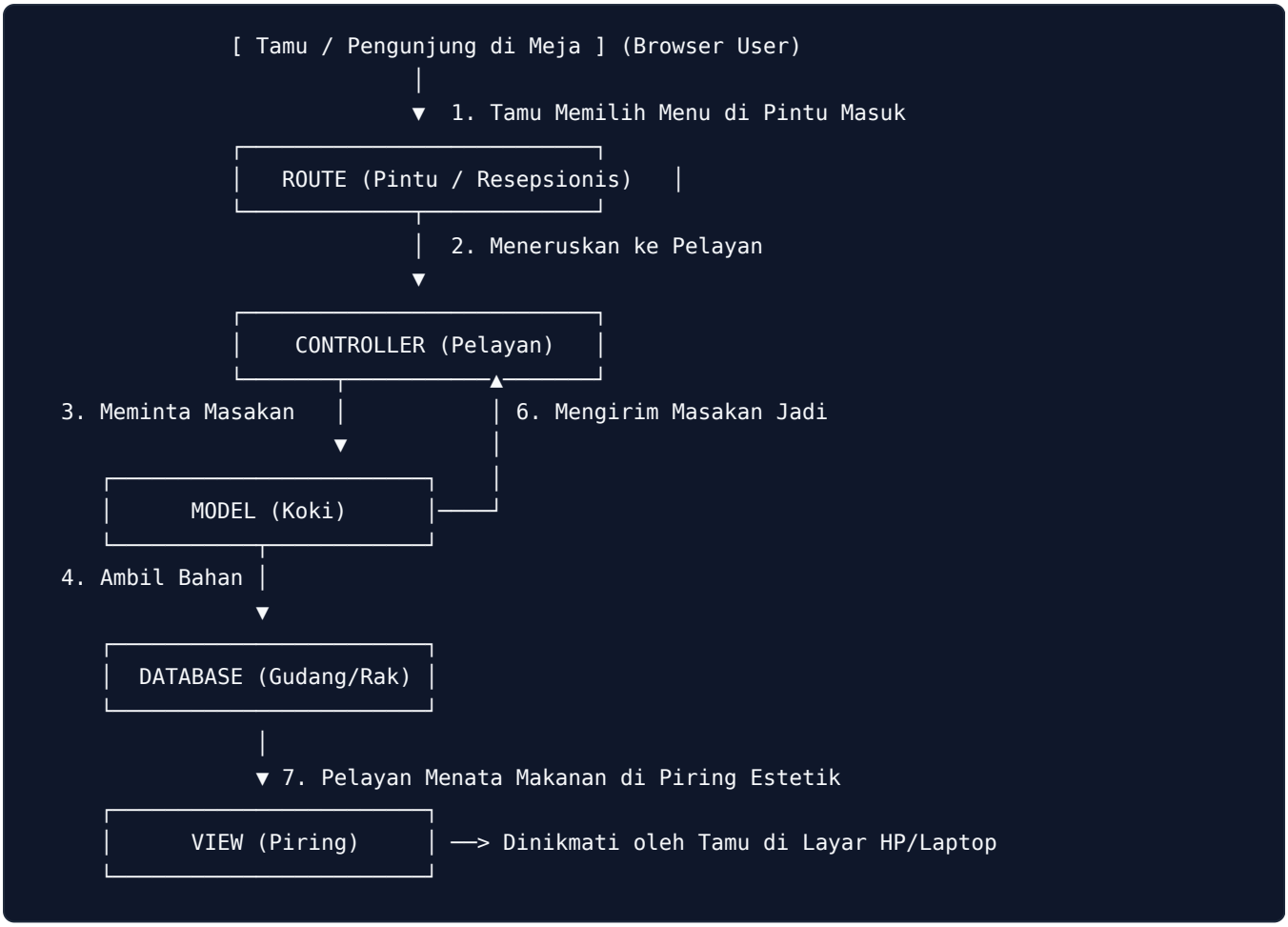
- **Cara Tradisional (PHP Native):** Kamu harus membuat batu bata sendiri dari tanah liat, mencampur semen sendiri, memasang pipa air sendiri, memasang kabel listrik dari nol, bahkan membuat engsel pintu sendiri. Jika ada kesalahan sambungan kabel, rumahmu bisa korsleting dan terbakar (banyak celah keamanan / bug).
- **Cara Modern (Laravel):** Kamu membeli rumah jadi dari pengembang ternama. Pondasinya sudah kokoh berstandar internasional, pipa air dan instalasi listrik sudah tertanam rapi di dinding, dan pintu-pintunya sudah memiliki kunci pintar. Kamu hanya tinggal mendekorasi cat tembok, mengisi perabotan ruang tamu, dan mengatur tata letak kamar sesuai kebutuhanmu.

> **Definisi Resmi untuk Penguji:**
> *"Laravel adalah framework aplikasi web berbasis PHP yang menyediakan struktur arsitektur siap pakai, standar keamanan tinggi (seperti proteksi CSRF, Enkripsi, Hashing), dan mempermudah developer membangun aplikasi menggunakan pola arsitektur MVC."*

---

2. Apa itu Pola MVC? (Analogi Restoran Bintang Lima)

Di dunia nyata, aplikasi Laravel bekerja persis seperti sebuah **Restoran Berbintang**:



1. **Route (Buku Menu / Pintu Masuk):**
   Ketika pembeli datang dan berkata *"Saya mau buka halaman /login"* atau *"Saya mau simpan tugas ke /tasks"*, Route yang pertama kali menyambut dan memeriksa buku catatan: *"Oh, menu pesanan ini harus saya serahkan ke Pelayan yang mana."*
2. **Controller (Pelayan):**
   Pelayan adalah pengatur lalu lintas. Pelayan tidak tahu cara menggoreng daging, tetapi pelayan tahu siapa yang harus dihubungi. Dia mencatat permintaan tamu, memeriksa apakah tamu membawa uang (validasi data), lalu berteriak ke bagian dapur (Model).
3. **Model (Koki):**
   Koki adalah ahli yang tahu aturan bahan baku. Dia tahu bahan apa yang boleh diambil dan bahan apa yang tidak boleh disentuh.
4. **Database SQLite (Lemari Bahan Makanan):**
   Tempat penyimpanan fisik semua data. Dalam proyek ini, SQLite adalah lemari arsip berupa **satu file buku tunggal** (`database/database.sqlite`), sehingga tidak perlu menyalakan mesin pendingin besar seperti MySQL/XAMPP.
5. **View (Piring Saji / Tata Hidang):**
   Setelah masakan matang dari Koki dan Pelayan, masakan itu tidak ditaruh sembarangan di meja. Masakan ditata rapi di piring cantik lengkap dengan garnish daun parsley (HTML + Tailwind CSS) agar nyaman dilihat oleh mata tamu.

BAB 2: PROSES PEMBUATAN DARI HULU KE HILIR

Jika penguji bertanya: *"Bagaimana langkah kamu membangun aplikasi ini dari awal sampai jadi?"*, jangan bingung! Jangan jawab acak. Jawablah dengan 6 langkah berurutan (*Software Development Life Cycle* mini) ini:

```
[Tahap 1: Migrasi] ──> [Tahap 2: Seeder] ──> [Tahap 3: Model]
                                                      |
[Tahap 6: View Blade] <── [Tahap 5: Controller] <─── [Tahap 4: Route]
```

1. **Tahap 1 - Merancang Database (Migrations):**
   Mendesain cetak biru tabel database (`users` , `categories` , `tasks`) beserta tipe kolom dan relasi kuncinya (Foreign Key).
2. **Tahap 2 - Menyiapkan Data Bawaan (Seeders):**
   Mengisi data master otomatis agar saat aplikasi baru dinyalakan, kategori dasar (`Sekolah` , `Pribadi` , `Pekerjaan`) sudah langsung ada tanpa perlu diinput manual.
3. **Tahap 3 - Menghubungkan Logika Tabel (Eloquent Models):**
   Membuat file Model PHP agar aplikasi bisa berbicara dengan database menggunakan bahasa objek, serta menentukan relasi (*One-to-Many* / *Belongs-To*).
4. **Tahap 4 - Menentukan Pintu Masuk (Routes & Middleware):**
   Menentukan URL apa saja yang ada di aplikasi dan menaruh satpam (*Middleware*) untuk membedakan tamu yang belum login (*guest*) dan pengguna yang sudah login (*auth*).
5. **Tahap 5 - Menulis Otak Logika (Controllers):**
   Membuat aturan bisnis: validasi data, simpan tugas, pencarian tugas, balik status selesai/belum, dan pencegahan manipulasi ID data orang lain (Anti-IDOR).
6. **Tahap 6 - Mendesain Antarmuka (Blade & Tailwind CSS):**
   Membuat halaman web yang responsif, modern, memiliki indikator warna prioritas, slider progres interaktif, dan konfirmasi aksi.

BAB 3: BEDAH FILE DAN KODE SECARA MENDETAIL

Mari kita bedah setiap file penting di proyek ini sampai ke akar-akarnya.

A. File Migrasi Database

1. `database/migrations/2024_01_01_000001_create_categories_table.php`

- **Tujuan File:** Membuat tabel `categories` di database untuk menyimpan data master jenis kategori tugas. Jika file ini dihapus, aplikasi tidak akan punya kategori dan tugas tidak tahu harus dikelompokkan ke mana.
- **Potongan Kode:**

```
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->timestamps();
});
```

- **Terjemahan Bahasa Manusia:**
  - `Schema::create('categories', ...)`: Komputer, tolong buatkan tabel baru bernama `categories`.
  - `$table->id()`: Buatkan kolom nomor urut otomatis (1, 2, 3...) yang menjadi Kunci Utama (*Primary Key*).
  - `$table->string('name')`: Buatkan kolom teks untuk menyimpan nama kategori (misal: "Sekolah").
  - `$table->timestamps()`: Buatkan otomatis dua kolom: `created_at` (kapan dibuat) dan `updated_at` (kapan terakhir diubah).
- **Analogi Nyata:** Seperti menyiapkan stopmap/folder arsip baru di lemari kantor berlabel "Kategori".
- **Poin Ujian:** Digunakan sebagai tabel referensi (*Parent Table*) bagi tabel transaksi `tasks`.

---

2. `database/migrations/2024_01_01_000002_create_tasks_table.php`

- **Tujuan File:** Membuat tabel transaksi utama `tasks` untuk menyimpan semua tugas pengguna.
- **Potongan Kode Penting:**

```
$table->foreignId('user_id')->constrained('users')->onDelete('cascade');
$table->foreignId('category_id')->constrained('categories')->onDelete('cascade');
$table->string('title', 255);
$table->text('description')->nullable();
$table->enum('priority', ['low', 'medium', 'high'])->default('medium');
$table->date('due_date')->nullable();
$table->unsignedTinyInteger('progress')->default(0);
$table->boolean('is_completed')->default(false);
```

🔍 Tanya Jawab Penguji: Apa Maksud `onDelete('cascade')`?

- **Arti Teknis:** Integritas referensial database (*Cascading Delete*).
- **Bahasa Manusia:** Jika akun seorang User dihapus dari database, maka secara otomatis **seluruh tugas-tugas milik user tersebut langsung ikut dibakar habis dari database**.
- **Analogi:** Jika seorang pemilik rumah pindah negara dan merobohkan rumahnya, semua barang di dalam rumah itu otomatis ikut hancur. Tidak ada barang mengambang tanpa pemilik (mencegah *Orphan Record*).

🔍 Tanya Jawab Penguji: Apa Bedanya `string` dan `text`?

- `string`: Batas maksimal 255 huruf. Cocok untuk teks pendek seperti `title` (judul).
- `text`: Daya tampung sangat besar (hingga ribuan huruf). Cocok untuk `description` (catatan rincian tugas).
- `nullable()`: Kolom ini boleh dikosongkan oleh user saat mengisi form.

B. File Seeder (Data Awal)

`database/seeders/CategorySeeder.php`

- **Tujuan File:** Memberikan suntikan data awal saat aplikasi baru dipasang.
- **Potongan Kode:**

```
$categories = ['Sekolah', 'Pribadi', 'Pekerjaan'];

foreach ($categories as $category) {
    Category::firstOrCreate(['name' => $category]);
}
```

- **Terjemahan Bahasa Manusia:**
  "Periksa tabel kategori. Apakah nama 'Sekolah' sudah ada? Kalau belum ada, buatkan baru. Kalau sudah ada, jangan buat lagi (abaikan)."
- **Analogi:** Seperti menyalakan HP baru pertama kali; pabrik sudah menyediakan aplikasi bawaan (Kamera, Jam, Kontak) agar HP bisa langsung dipakai tanpa layar kosong melompong.

- **Poin Ujian:** Penggunaan `firstOrCreate()` bersifat **Idempotent** (aman dijalankan berkali-kali tanpa menghasilkan duplikasi data ganda).

C. File Model & Relasi Eloquent

Model adalah perantara suci antara kode PHP dan tabel database. Dengan Model, kita tidak perlu lagi menulis query SQL panjang seperti `SELECT * FROM tasks WHERE...`.

1. `app/Models/User.php`

- **Potongan Kode:**

```
protected $fillable = ['name', 'email', 'password'];

public function tasks()
{
    return $this->hasMany(Task::class);
}
```

- **Poin Ujian Relasi:**
  `hasMany(Task::class)` artinya **Satu User memiliki Banyak Tugas (Relasi One-to-Many)**.

---

2. `app/Models/Task.php`

- **Potongan Kode:**

```
protected $fillable = [
    'user_id',
    'category_id',
    'title',
    'description',
    'priority',
    'due_date',
    'progress',
    'is_completed',
];

protected $casts = [
    'is_completed' => 'boolean',
    'progress' => 'integer',
    'due_date' => 'date',
];

public function user()
{
    return $this->belongsTo(User::class);
}

public function category()
{
    return $this->belongsTo(Category::class);
}
```

🔍 Tanya Jawab Penguji: Apa Fungsi `$fillable` dan Bahayanya Jika Tidak Ada?

- **Arti Teknis:** Whitelist proteksi dari celah keamanan **Mass Assignment Vulnerability**.
- **Bahasa Manusia:** `$fillable` adalah daftar tamu VIP. Hanya kolom yang namanya tertulis di dalam kurung siku `$fillable` yang diizinkan untuk diisi secara massal lewat form input web.
- **Bahayanya:** Bayangkan jika tabel users punya kolom `is_admin`. Jika tidak ada `$fillable`, seorang hacker bisa menambahkan input rahasia `<input name="is_admin" value="1">` di browser, dan sistem akan langsung menjadikan hacker tersebut Admin tertinggi aplikasi!
- **Analogi:** Satpam gedung yang memegang daftar nama orang yang boleh masuk lift khusus.

🔍 Tanya Jawab Penguji: Apa Fungsi `$casts`?

- Di database SQLite, nilai `is_completed` sebenarnya disimpan sebagai angka integer `0` atau `1`.
- Berkat `$casts = ['is_completed' => 'boolean']`, Laravel otomatis mengubah angka `0` menjadi `false` dan angka `1` menjadi `true` di level PHP, sehingga kita bisa langsung menulis logika `if ($task->is_completed)`.

D. File Routing & Satpam Akses (Middleware)

`routes/web.php`

- **Tujuan File:** Buku panduan alamat URL aplikasi.
- **Potongan Kode:**

```
Route::middleware('guest')->group(function () {
    Route::get('/login', [AuthController::class, 'showLoginForm'])->name('login');
    Route::post('/login', [AuthController::class, 'login']);
    Route::get('/register', [AuthController::class, 'showRegisterForm'])->name('register')
    Route::post('/register', [AuthController::class, 'register']);
});

Route::middleware('auth')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

    Route::get('/', [TaskController::class, 'index'])->name('tasks.index');
    Route::post('/tasks', [TaskController::class, 'store'])->name('tasks.store');
    Route::put('/tasks/{id}', [TaskController::class, 'update'])->name('tasks.update');
    Route::patch('/tasks/{id}/toggle', [TaskController::class, 'toggle'])->name('tasks.tog
    Route::patch('/tasks/{id}/progress', [TaskController::class, 'updateProgress'])->name(
    Route::delete('/tasks/{id}', [TaskController::class, 'destroy'])->name('tasks.destroy'
});
```

🔍 Tanya Jawab Penguji: Apa Bedanya Middleware `guest` dan `auth`?

- **Middleware `guest` (Tamu):** Hanya boleh dimasuki oleh orang yang **BELUM** login. Jika orang yang sudah login mencoba membuka `/login`, satpam langsung menendang dia kembali ke halaman utama dashboard.
- **Middleware `auth` (Terotentikasi):** Pintu VIP yang **HANYA** boleh dimasuki oleh user yang sudah berhasil login (memiliki tiket sesi aktif). Orang luar yang belum login akan langsung diusir ke halaman `/login`.

E. File Controller (Otak Logika Aplikasi)

1. `app/Http/Controllers/AuthController.php`

- **Proses Login & Session Fixation:**

```
if (Auth::attempt($credentials, $request->boolean('remember'))) {
    $request->session()->regenerate();
    return redirect()->intended(route('tasks.index'));
}
```

- **Poin Ujian Session Fixation:** Kenapa harus ada `$request->session()->regenerate()`? *Untuk membuang tiket nomor antrean lama dan mencetak tiket baru yang aman setelah user berhasil membuktikan identitasnya. Ini menutup celah hacker yang mencoba mencuri sesi login pengguna.*

---

2. `app/Http/Controllers/TaskController.php`

Ini adalah file paling penting dalam ujian UKK Anda!

A. Method `index()` (Menampilkan & Memfilter Tugas)

```
$tasks = auth()->user()->tasks()
    ->with('category')
    ->when($request->filled('search'), function ($query) use ($request) {
        $query->where('title', 'like', '%' . $request->search . '%');
    })
    ->when($request->filled('status'), function ($query) use ($request) {
        if ($request->status === 'completed') {
            $query->where('is_completed', true);
        } elseif ($request->status === 'pending') {
            $query->where('is_completed', false);
        }
    })
    ->when($request->filled('category_id'), function ($query) use ($request) {
        $query->where('category_id', $request->category_id);
    })
    ->latest()
    ->get();
```

🌟 Bedah Khusus: Apa itu Masalah N+1 dan Solusi `with('category')`?

- **Kasus Tanpa `with('category')` (N+1 Problem):**
  Bayangkan kamu punya 100 tugas.
  1. Kueri ke-1: Mengambil 100 data tugas (`SELECT * FROM tasks`).
  2. Saat menampilkan nama kategori di looping view: Laravel terpaksa bolak-balik ke database sebanyak 100 kali lagi (`SELECT * FROM categories WHERE id = ...`).
     Total kueri: 1 + 100 = 101 Kueri Database! Website jadi lambat dan boros memori!
- **Solusi Eager Loading `with('category')`:**
  Laravel hanya menjalankan **2 kueri**:
  1. `SELECT * FROM tasks WHERE user_id = 1`
  2. `SELECT * FROM categories WHERE id IN (1, 2, 3)`
     Lalu Laravel mencocokkan datanya di memori RAM internal. Ringan, cepat, dan profesional!

🌟 Bedah Khusus: Apa Keunggulan Eloquent `when()`?

- Menggantikan percabangan `if (...)` konvensional yang berantakan. Query hanya akan ditambahkan klausa `where` jika parameter input benar-benar diisi oleh user di formulir filter.

B. Method `store()` (Menyimpan Tugas Baru)

```
$validated = $request->validate([
    'title' => 'required|string|max:255',
    'description' => 'nullable|string',
    'priority' => 'required|in:low,medium,high',
    'due_date' => 'nullable|date',
    'category_id' => 'required|exists:categories,id',
    'progress' => 'nullable|integer|min:0|max:100',
]);

auth()->user()->tasks()->create($validated);
```

🌟 Bedah Khusus: Kenapa DILARANG Menaruh `<input type="hidden" name="user_id">` di Form HTML?

- **Jawaban Tegas ke Penguji:**
  "Karena input HTML di sisi browser sangat mudah dimanipulasi melalui Inspect Element (DevTools). Jika ada hidden input `user_id`, seorang user bisa mengganti angka ID miliknya menjadi ID orang lain untuk memalsukan kepemilikan data."
- Dengan perintah `auth()->user()->tasks()->create($validated)`, Laravel otomatis menyisipkan `user_id` asli dari sesi login pengguna yang terotentikasi di server. Aman 100%!

C. Method `toggle()` & `updateProgress()` (Anti-IDOR & Negasi Boolean)

```
public function toggle($id)
{
    $task = auth()->user()->tasks()->findOrFail($id);

    $newCompleted = !$task->is_completed;

    $task->update([
        'is_completed' => $newCompleted,
        'progress' => $newCompleted ? 100 : 0,
    ]);

    return redirect()->back()->with('success', 'Status tugas berhasil diperbarui!');
}
```

🌟 Bedah Khusus: Mengapa `auth()->user()->tasks()->findOrFail($id)` Disebut Anti-IDOR?

- **Apa itu IDOR?** *Insecure Direct Object Reference* (Celah keamanan di mana pengguna bisa mengubah data milik orang lain hanya dengan mengganti nomor ID pada URL).
- Jika kita menulis: `Task::findOrFail($id)`, sistem mencari ID tersebut di seluruh database tanpa peduli punya siapa.

- Tetapi dengan `auth()->user()->tasks()->findOrFail($id)`, pencarian **dikunci (scoped)** hanya di dalam daftar tugas milik user yang sedang login. Jika user mencoba mengakses ID milik orang lain, sistem akan otomatis melempar pesan **404 Not Found (Data Tidak Ditemukan)**!

🌟 Bedah Khusus: Apa Fungsi Tanda Seru `!$task->is_completed`?

- Tanda seru (`!`) adalah operator logika **NOT (Negasi / Pembalik)**.
- Jika `is_completed` bernilai `false`, maka `!false` menghasilkan `true`.
- Jika `is_completed` bernilai `true`, maka `!true` menghasilkan `false`.
- Seperti saklar lampu: ditekan sekali menyala, ditekan lagi mati.

F. File Tampilan (Blade Views)

1. `resources/views/layouts/app.blade.php`

- **Kenapa ada `@csrf` di setiap form non-GET?**
  CSRF (Cross-Site Request Forgery) adalah serangan di mana situs jahat luar mengirimkan formulir palsu atas nama user yang sedang login. Token `@csrf` bertindak seperti **stempel cap basah resmi** yang diverifikasi oleh Laravel. Formulir tanpa stempel ini langsung ditolak dengan kode error `419 Page Expired`.
- **Kenapa ada Konfirmasi Keluar?**
  Atribut `onsubmit="return confirm('Apakah Anda yakin ingin keluar dari akun?');"` memunculkan jendela konfirmasi dialog kecil di browser agar user tidak sengaja keluar saat sedang asyik bekerja.

2. `resources/views/tasks/index.blade.php`

- **Kenapa ada `@method('PATCH')` dan `@method('DELETE')`?**
  Formulir HTML standar di browser dunia manapun **hanya mengenal 2 metode HTTP: GET dan POST**. Agar kita bisa mengikuti standar arsitektur RESTful API modern (di mana ubah sebagian status pakai `PATCH` dan hapus data pakai `DELETE`), Laravel menyediakan teknik **Method Spoofing** lewat perintah `@method('PATCH')`.

BAB 4: ALUR PERJALANAN DATA (STUDI KASUS NYATA)

Mari kita bayangkan apa yang terjadi di kabel jaringan komputer saat pengguna melakukan aksi:

Kasus 1: Pengguna Menyimpan Tugas Baru

```
[User Mengisi Form] ──> Klik "Simpan Tugas"
                            | (Kirim HTTP POST /tasks + Data + Token CSRF)
                            ▼
                      [routes/web.php]
                            | (Cek Middleware auth: User Lolos)
                            ▼
                [TaskController@store]
                            | 1. Validasi teks judul, kategori, bobot prioritas
                            | 2. auth()->user()->tasks()->create($validated)
                            ▼
                    [Database SQLite]
                            | Menyimpan baris baru: ID=5, user_id=1, title="Belajar UKK"
                            ▼
                    [Redirect ke /] ──> Browser menampilkan tugas baru paling atas
                                        beserta alert hijau "Tugas berhasil ditambahkan!"
```

Kasus 2: Pengguna Menggeser Slider Progres Mentok Kanan (100%)

1. Pengguna menggeser knob slider ke angka `100` atau mengklik tombol bulat bertuliskan **`100`**.
2. Fungsi JavaScript `onSliderLiveChange(taskId, 100)` langsung berjalan seketika di browser:
   - Mengubah badge menjadi hijau zamrud `100%`.
   - Mencoret teks judul tugas (*line-through*).
   - Mencentang kotak checkbox tugas.
3. Fungsi `onSliderCommit(taskId, 100)` mengirim paket data di latar belakang (*Fetch API / AJAX*) menggunakan metode `PATCH` ke URL `/tasks/5/progress`.
4. Controller di server menerima angka 100, mengenali bahwa `progress >= 100`, lalu mengubah `is_completed = true` di database SQLite.
5. Server membalas dengan status JSON `{ success: true }`, dan counter ringkasan tugas selesai di bagian atas bertambah tanpa perlu me-reload halaman!

Kasus 3: Pengguna Nakal Mencoba Membobol Tugas Orang Lain (Simulasi IDOR)

1. User A (ID: 1) mencoba mengirim perintah hapus tugas ID 99 yang sebenarnya adalah milik User B (ID: 2):

```
DELETE /tasks/99
```

2. Request sampai di `TaskController@destroy($id)`.
3. Server mengeksekusi kueri:

```
SELECT * FROM tasks WHERE user_id = 1 AND id = 99
```

4. Karena tugas 99 adalah milik User 2, hasil kueri di atas adalah **KOSONG (NULL)**.
5. Method `findOrFail()` langsung membunyikan alarm dan menghentikan proses, lalu melempar respon HTTP **404 Not Found**.
6. **Hasil:** Data User B tetap aman, tidak tersentuh dan tidak terhapus sedikitpun!

BAB 5: KAMUS ISTILAH ASING KE BAHASA MANUSIA

Gunakan tabel ini sebagai kamus hafalan saat penguji menguji istilah-istilah teknis:

Istilah Programmer	Bahasa Sehari-Hari / Analogi	Makna Teknis di Aplikasi
CRUD	Bikin, Liat, Edit, Hapus	*Create, Read, Update, Delete* (4 operasi dasar pengolahan data tabel).
Scoped Query	Mengunci area pencarian	Membatasi kueri hanya pada milik user aktif via `auth()->user()->tasks()`.
IDOR	Mengintip & membajak data tetangga	*Insecure Direct Object Reference* (Celah keamanan menebak ID orang lain).
Eager Loading	Sekali jalan bawa semua	Mengambil data tugas beserta relasi kategorinya sekaligus via `with()`.
N+1 Problem	Bolak-balik warung 100 kali	Kueri berulang-ulang yang bikin database lemot karena tidak pakai Eager Loading.
Mass Assignment	Menyelundupkan barang di bagasi	Kerentanan memasukkan input kolom berbahaya tanpa filter `$fillable`.
CSRF	Stempel cap basah anti-pemalsuan	Token rahasia `@csrf` untuk memastikan form benar-benar dikirim dari web resmi.
Method Spoofing	Menyamar pakai jaket	Trik `@method('PATCH')` agar form HTML biasa bisa menjalankan aksi PATCH/DELETE.
Nullable	Boleh kosong	Kolom database yang tidak wajib diisi (seperti kolom catatan/deskripsi).
Cascade Delete	Efek kartu domino yang runtuh	Jika data induk (User) dihapus, seluruh data anaknya (Tasks) terhapus otomatis.
Bcrypt / Hash	Daging digiling jadi kornet	Mengacak password menjadi kode satu arah yang tidak bisa dipecahkan kembali.
Session Fixation	Membakar tiket lama usai gerbang terbuka	Mengacak session ID baru via `session()->regenerate()` pasca login berhasil.
Boolean	Saklar lampu (Cuma On atau Off)	Tipe data logika yang hanya bernilai dua kemungkinan: `true` (benar/selesai) atau `false` (salah/belum).

🎓 PESAN MENTAL UNTUK HARI UJIAN:

> *"Uji Kompetensi Keahlian bukanlah tentang menghafal jutaan baris kode di luar kepala, melainkan tentang* ***memahami mengapa baris kode itu ada di sana*** *dan* ***bagaimana kamu melindungi data pengguna****. Percaya diri, jelaskan dengan tenang menggunakan alur logika dan analogi di atas, dan buktikan bahwa kamu adalah calon Software Engineer masa depan yang kompeten!"* **Selamat Berjuang & Sukses UKK!** 🚀